

Machine Learning and Artificial Neural Networks: Problem Set

Introduction to Electrical Engineering — Unit Companion

Problems 1–8 are conceptual, 9–18 analytical, and 19–24 numerical (Python). Solutions begin on page 5. Throughout, networks use the biased form $\mathbf{h}^{(l)} = f_a(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)})$ with ReLU hidden activations and a softmax output unless stated otherwise.

Conceptual Problems

Problem 1 (Deterministic vs. probabilistic). A microcontroller opens a water valve when a thermistor reads above 30°C. A second system must halt a machine when it hears the spoken word “stop.” Explain why the first task should be hard-coded while the second calls for machine learning. What property of the input-to-output mapping makes the difference?

Problem 2 (Learning paradigms). Define supervised learning. Name the two other principal categories of machine learning and state, in one sentence each, how they differ from it. Which paradigm does backpropagation-based classifier training belong to, and why?

Problem 3 (Deep networks and abstraction). What does the “deep” in deep learning refer to? Describe qualitatively what happens to the *kind* of decision being made as information passes through successive hidden layers of a digit-recognition network.

Problem 4 (Perceptron vs. sigmoid neuron). A perceptron and a sigmoid neuron have identical weights and bias. (a) In what limit do their outputs agree? (b) Explain why training a multi-layer network of perceptrons by small weight adjustments fails, and how the sigmoid neuron fixes this. (c) Give the criterion typically used to convert a continuous sigmoid output into a yes/no decision.

Problem 5 (Output encoding). For 10-digit recognition, why does a 10-neuron output layer (one per digit) typically work better than a 4-neuron binary-encoded output, even though 4 bits suffice to represent 10 classes? Frame your answer in terms of what the output neurons must compute from hidden-layer pattern detectors.

Problem 6 (Role of the bias). In $y = f(\mathbf{w} \cdot \mathbf{x} + b)$: (a) what is the effect of a large positive b ? A large negative b ? (b) Show that a perceptron with threshold T (“fire if $\sum w_i x_i > T$ ”) is equivalent to a perceptron with bias $b = -T$ and threshold 0. (c) In the delta-rule update, the bias is treated as a weight on what input?

Problem 7 (Holdout method). A classmate trains a model, evaluates it on the test set, adjusts the hidden layer width to improve the test accuracy, re-evaluates, and repeats ten times, then reports the best test accuracy. Name the standard 60/20/20 data split, explain each split’s job, and explain what is wrong with the classmate’s procedure and why the reported number is optimistic.

Problem 8 (Naive classifier). A dataset contains 989 images of fields and 11 of dogs. A model outputs “field” for every input. (a) What is its accuracy? (b) Why is this model useless despite the number in (a)? (c) Which single reported metric from this unit most clearly exposes the failure, and what value does it take for the “dog” class?

Analytical Problems

Problem 9 (Design a perceptron gate). Find weights (w_1, w_2) and bias b so that a single perceptron (Eq. $y = 1$ iff $\mathbf{w} \cdot \mathbf{x} + b > 0$) implements the OR function. Verify all four rows of the truth table, and give the equation of the decision boundary line.

Problem 10 (XOR impossibility). (a) Prove that no single perceptron implements XOR. (Hint: write the four inequalities the weights would have to satisfy and derive a contradiction.) (b) Express XOR as a two-layer composition of AND, OR, and NAND, and verify it on the input $(1, 1)$ using the gate parameters of Problem 9 and the standard AND $(1, 1, b = -1.5)$ and NAND $(-1, -1, b = 1.5)$ parameters.

Problem 11 (Forward solve by hand). A network has $N = 2$ inputs, one hidden layer of $A = 2$ ReLU nodes, and $M = 2$ softmax outputs:

$$\mathbf{W}^{(1)} = \begin{bmatrix} 2 & 1 \\ -1 & 1 \end{bmatrix}, \mathbf{b}^{(1)} = \begin{bmatrix} -1 \\ 0 \end{bmatrix}, \mathbf{W}^{(2)} = \begin{bmatrix} 1 & -2 \\ 0 & 1 \end{bmatrix}, \mathbf{b}^{(2)} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \mathbf{x} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

Compute $\mathbf{z}^{(1)}$, $\mathbf{h}^{(1)}$, $\mathbf{z}^{(2)}$, the softmax probabilities, and the predicted class.

Problem 12 (Softmax properties). For raw outputs $\mathbf{y} = [2, 0, -1]^\top$: (a) compute the softmax probabilities to three decimals and verify they sum to 1. (b) Show algebraically that subtracting a constant c from every element of $\mathbf{y}$ leaves the softmax unchanged. (c) Why do numerical implementations exploit (b), and what value of c do they use?

Problem 13 (One backpropagation step). For the network and input of Problem 11, take the correct label $\hat{\mathbf{y}} = [0, 1]^\top$ and $\alpha = 0.1$. Compute the error vector $\boldsymbol{\varepsilon}$, the output delta $\boldsymbol{\delta}^{(2)}$, the updates $\Delta \mathbf{W}^{(2)}$ and $\Delta \mathbf{b}^{(2)}$, and the hidden delta $\boldsymbol{\delta}^{(1)} = \{[\mathbf{W}^{(2)}]^\top \boldsymbol{\delta}^{(2)}\} \odot f'_a(\mathbf{z}^{(1)})$. Comment on any zero entries.

Problem 14 (Parameter counting). Counting weights *and* biases: (a) how many trainable parameters does the 5×5 digit network have ($N = 25$, $A = 20$, $M = 5$)? (b) An MNIST-sized network with the same hidden layer ($N = 784$, $A = 20$, $M = 10$)? (c) The same but with the hidden layer matched to the input ($A = 784$)? (d) For fixed $N = 784$ and $M = 10$, write the parameter count as a function of a single hidden width A and determine the largest A for which the float32 model fits in 256 kB of RAM.

Problem 15 (RMS feature). Two 2-Hz-sampled windows of x -axis acceleration read (in m/s^2):

$$x_{\text{active}} = [-3, 4, -4, 3], \quad x_{\text{idle}} = [0.2, -0.2, 0.2, -0.2].$$

(a) Compute the mean and RMS of each window. (b) Which statistic separates the two windows, and why does the other fail? (c) The gesture classifier reduces 375 raw values to 33 features (11 per axis). List the 11 features per axis and state the two “problems with deep learning” this reduction addresses.

Problem 16 (Two-class confusion matrix). A binary classifier’s validation results: actual-positive samples: 38 predicted positive, 17 predicted negative; actual-negative: 3 predicted positive, 42 predicted negative. Identify TP, FN, FP, TN, then compute ACC, TPR, TNR, PPV, and F_1 to three decimals. Which type of error (I or II) dominates?

Problem 17 (Multi-class metrics). Using the 1000-sample gesture confusion matrix (rows = actual, columns = predicted, order Circle/Idle/Left-Right/Up-Down):

$$\begin{bmatrix} 205 & 10 & 1 & 46 \\ 6 & 199 & 0 & 32 \\ 9 & 17 & 223 & 34 \\ 21 & 8 & 3 & 186 \end{bmatrix}$$

(a) compute the total accuracy; (b) treating *Up-Down* as positive, find TP, FN, FP, TN and compute ACC, TPR, PPV, and F_1 ; (c) explain, from the matrix structure, why Up-Down is the weakest class.

Problem 18 (Deployment budget). A classifier with 33 inputs, hidden layers of 20 and 10 nodes, and 4 outputs runs on a 64-MHz Cortex-M4 sustaining 1 `float32` MAC per cycle. (a) Compute MACs per inference and total parameters (weights + biases). (b) Compute the weight memory in `float32` and in `int8`. (c) Estimate the network evaluation time and compare it to the 2-s sensing window; what pipeline stage actually dominates?

Numerical (Python) Problems

Problem 19 (Activation functions). Implement ReLU, sigmoid, and tanh in NumPy and plot all three on $x \in [-6, 6]$ in one figure. Numerically verify the identity $\tanh(x) = 2\sigma(2x) - 1$ at 100 random points, and verify that the ReLU derivative mask ($\mathbf{x} > 0$) matches $(f(x + \epsilon) - f(x - \epsilon))/2\epsilon$ for $\epsilon = 10^{-6}$ away from the origin.

Problem 20 (Train the digit classifier). Using the training function of the companion script (Listing 1 of the encyclopedia), train the 5×5 digit network for 10^3 epochs at each of $\alpha \in \{0.001, 0.005, 0.01, 0.05, 0.5\}$, plotting all five learning curves on log-log axes together. Report the final cost for each α and identify which learning rates are in the stable regime.

Problem 21 (Hidden-width sweep). At fixed $\alpha = 0.01$, sweep the hidden-layer width over $A = 5, 10, 20$, and 25, and plot the four learning curves. Then add a second hidden layer of equal width for $A = 20$ and compare against the single-layer curve. Does depth help on this 5-image problem? Explain in terms of §9 of the encyclopedia (model fit).

Problem 22 (Feature extraction pipeline). Generate synthetic 2-s, 62.5-Hz accelerometer windows for two classes: “wave” ($x(t) = 6 \sin(2\pi \cdot 1 \text{ Hz} \cdot t) + \text{noise}$) and “idle” (noise only), 50 windows each with Gaussian noise $\sigma = 0.4 \text{ m/s}^2$. For each window compute (i) RMS and (ii) the frequency of the largest PSD peak. Scatter-plot the two features with class colors. Are the classes linearly separable in feature space?

Problem 23 (Confusion matrix from scratch). Write a Python function `confusion(actual, predicted, K)` that returns the $K \times K$ confusion matrix, and a function returning per-class (ACC, TPR, TNR, PPV, F_1). Validate both against the worked Idle-class numbers of the encyclopedia (ACC 0.927, TPR 0.840, TNR 0.954, PPV 0.850, F_1 0.845) by constructing prediction arrays consistent with the gesture confusion matrix.

Problem 24 (Early stopping experiment). Augment the 5-digit training set: generate 40 noisy copies of each digit (flip each pixel with probability 0.08) and split 60/20/20 into train/validation/test. Train at $\alpha = 0.01$ while recording training *and* validation classification accuracy each epoch. Plot both curves, identify the epoch at which validation accuracy stops improving, apply early stopping there, and report final test accuracy. Which overfitting countermeasure from the encyclopedia does this implement?

Solutions

Solution 1. The thermometer task has a deterministic input–output mapping: a given reading always corresponds to the same correct action, so a fixed rule (if $T > 30$) is complete and optimal. Speech is probabilistic: every utterance of “stop” yields a different waveform (speaker, pitch, speed, noise), so no finite hand-written rule covers all cases; we instead give an algorithm many labeled examples and let it tune its own parameters to approximate the mapping statistically.

Solution 2. Supervised learning trains on examples whose correct labels are known in advance, adjusting parameters until predictions match the labels. *Unsupervised* learning extracts structure from unlabeled data (no correct answers provided). *Reinforcement* learning has an agent learn from rewards received while interacting with an environment (feedback is a scalar reward, not the correct answer). Backpropagation-based classifier training is supervised: the error $\hat{\mathbf{y}} - \mathbf{y}$ that drives every update requires the known correct label $\hat{\mathbf{y}}$.

Solution 3. “Deep” refers to the number of layers of units between the input and output layers; training a network with at least one hidden layer is deep learning. Early hidden layers make decisions about low-level, concrete patterns (e.g., is a particular short stroke present in these pixels); later layers combine those outputs into decisions about progressively more abstract items (arcs, digit shapes), until the output layer decides among whole classes.

Solution 4. (a) For large $|z| = |\mathbf{w} \cdot \mathbf{x} + b|$: $\sigma(z) \rightarrow 1$ for large positive z and $\rightarrow 0$ for large negative z , matching the perceptron. (b) Learning by gradient descent requires small weight changes to cause small output changes. A perceptron’s step output can flip 0→1 under an infinitesimal weight change; the flip propagates through downstream layers and can radically change many other classifications in a way that is complex to precalculate — you fix one “3” and break other digits. The sigmoid smooths the transition so outputs vary continuously with weights, making incremental error reduction possible. (c) Threshold the output, e.g., “yes” if the value exceeds 0.5.

Solution 5. With one neuron per class, the “0” output neuron only needs to apply large weights to the hidden neurons that detect zero-like patterns (e.g., the four arcs of a 0) and weaken the rest — a single weighted vote over pattern detectors. With a 4-bit binary encoding, each output bit is shared by several digits, so the network must effectively both determine the digit *and* convert it to binary — two computations tangled into the same weights, which is harder to learn.

Solution 6. (a) Large positive b : the neuron fires on most inputs; large negative b : it rarely fires. (b) “ $\sum w_i x_i > T$ ” $\iff$ “ $\sum w_i x_i - T > 0$ ” $\iff$ “ $\mathbf{w} \cdot \mathbf{x} + b > 0$ ” with $b = -T$. (c) A constant input of 1: since $\Delta \mathbf{b}^{(l)} = \alpha \delta^{(l)}$, the bias updates exactly as a weight whose input h is always 1 ($\Delta W = \alpha \delta h$ with $h = 1$).

Solution 7. Training set ($\sim 60\%$): used to update the parameters. Validation set ($\sim 20\%$): used to evaluate on unseen data while tweaking hyperparameters and repeating training. Test set ($\sim 20\%$): one final evaluation of the finished model. The classmate used the test set to *select* a hyperparameter (hidden width), i.e., as a validation set. After ten adapt-and-peek rounds, the best-of-ten test score partly reflects lucky alignment with that particular test set rather than true generalization, so the reported accuracy is biased upward. The fix: tune on validation, report a single final run on the untouched test set.

Solution 8. (a) $989/1000 = 98.9\%$. (b) It has learned nothing — it cannot detect a dog under any circumstance; the accuracy merely mirrors the class imbalance. (c) The F1 score of the minority class: with $TP = 0$ for “dog,” $F_1 = 2TP/(2TP + FP + FN) = 0$. (The F1 *average* across classes similarly collapses, e.g., 0.247 in the encyclopedia’s imbalanced example, while accuracy reads 0.973.)

Solution 9. Take $w_1 = w_2 = 1$, $b = -0.5$. Truth table of $\mathbf{w} \cdot \mathbf{x} + b$: $(0, 0) : -0.5 \Rightarrow 0$; $(0, 1) : +0.5 \Rightarrow 1$; $(1, 0) : +0.5 \Rightarrow 1$; $(1, 1) : +1.5 \Rightarrow 1$. Correct OR. Decision boundary: $x_1 + x_2 = 0.5$ — any input with at least one 1 lies on the positive side. (Any positive $w_1 = w_2$ with $0 < -b < w_1$ works.)

Solution 10. (a) Suppose (w_1, w_2, b) implements XOR. The truth table requires: $(0, 0) : b \leq 0$; $(1, 1) : w_1 + w_2 + b \leq 0$; $(0, 1) : w_2 + b > 0$; $(1, 0) : w_1 + b > 0$. Adding the last two: $w_1 + w_2 + 2b > 0$, so $w_1 + w_2 + b > -b \geq 0$, contradicting the $(1, 1)$ condition. No solution exists — XOR’s positive points $(0, 1), (1, 0)$ cannot be separated from $(0, 0), (1, 1)$ by one line (Minsky–Papert, 1969). (b) $\text{XOR} = \text{AND}(\text{OR}(x_1, x_2), \text{NAND}(x_1, x_2))$. At $(1, 1)$: OR gives $1 + 1 - 0.5 = 1.5 > 0 \Rightarrow 1$; NAND gives $-1 - 1 + 1.5 = -0.5 \leq 0 \Rightarrow 0$; AND of $(1, 0)$: $1 + 0 - 1.5 = -0.5 \leq 0 \Rightarrow 0$. Correct ($1 \oplus 1 = 0$).

Solution 11. $\mathbf{z}^{(1)} = \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)} = [2+1-1, -1+1+0]^\top = [2, 0]^\top$. ReLU: $\mathbf{h}^{(1)} = [2, 0]^\top$ (note $\max(0, 0) = 0$). $\mathbf{z}^{(2)} = \mathbf{W}^{(2)}\mathbf{h}^{(1)} + \mathbf{b}^{(2)} = [2-0, 0+1]^\top = [2, 1]^\top$. Softmax: $e^2 = 7.389$, $e^1 = 2.718$, sum = 10.107, so $\mathbf{y} = [0.731, 0.269]^\top$. Predicted class: 1.

Solution 12. (a) $e^2 = 7.389$, $e^0 = 1$, $e^{-1} = 0.368$; sum = 8.757. Probabilities: $[0.844, 0.114, 0.042]$; sum = 1.000. (b) $\frac{e^{y_m - c}}{\sum_{m'} e^{y_{m'} - c}} = \frac{e^{-c} e^{y_m}}{e^{-c} \sum_{m'} e^{y_{m'}}} = \frac{e^{y_m}}{\sum_{m'} e^{y_{m'}}}$: the factor e^{-c} cancels. (c) e^y overflows float for large raw outputs; choosing $c = \max_m y_m$ makes the largest exponent exactly $e^0 = 1$ and all others ≤ 1 , so overflow is impossible while the result is unchanged.

Solution 13. From Solution 11: $\mathbf{y} = [0.731, 0.269]^\top$, $\mathbf{h}^{(1)} = [2, 0]^\top$, $\mathbf{z}^{(1)} = [2, 0]^\top$. $\boldsymbol{\varepsilon} = \hat{\mathbf{y}} - \mathbf{y} = [-0.731, +0.731]^\top = \boldsymbol{\delta}^{(2)}$.

$$\Delta \mathbf{W}^{(2)} = 0.1 \begin{bmatrix} -0.731 \\ 0.731 \end{bmatrix} \begin{bmatrix} 2 & 0 \end{bmatrix} = \begin{bmatrix} -0.146 & 0 \\ 0.146 & 0 \end{bmatrix}, \quad \Delta \mathbf{b}^{(2)} = \begin{bmatrix} -0.0731 \\ 0.0731 \end{bmatrix}.$$

Hidden delta: $[\mathbf{W}^{(2)}]^\top \boldsymbol{\delta}^{(2)} = \begin{bmatrix} 1 & 0 \\ -2 & 1 \end{bmatrix} \begin{bmatrix} -0.731 \\ 0.731 \end{bmatrix} = [-0.731, 2.193]^\top$; the ReLU mask is $f'_a(\mathbf{z}^{(1)}) = [(2 > 0), (0 > 0)] = [1, 0]$, so $\boldsymbol{\delta}^{(1)} = [-0.731, 0]^\top$. Zeros: the second column of $\Delta \mathbf{W}^{(2)}$ vanishes because hidden node 2 carried no signal ($h_2 = 0$), and $\delta_2^{(1)} = 0$ because that node’s ReLU was inactive (by the $z = 0$ convention, derivative 0) — an inactive neuron neither takes nor assigns blame this step. Its bias $b_2^{(1)}$ therefore also freezes.

Solution 14. Using $P_{\text{layer}} = N_{\text{out}}N_{\text{in}} + N_{\text{out}}$: (a) $(20 \cdot 25 + 20) + (5 \cdot 20 + 5) = 520 + 105 = 625$. (b) $(20 \cdot 784 + 20) + (10 \cdot 20 + 10) = 15,700 + 210 = 15,910$. (c) $(784 \cdot 784 + 784) + (10 \cdot 784 + 10) = 615,440 + 7,850 = 623,290$. (d) $P(A) = (784A + A) + (10A + 10) = 795A + 10$. Memory = $4P$ bytes $\leq 262,144$ B $\Rightarrow P \leq 65,536 \Rightarrow A \leq (65,536 - 10)/795 = 82.4$, so $A_{\text{max}} = 82$ (and in practice less, since RAM must also hold activations, stack, and firmware data).

Solution 15. (a) Active window: $\text{mean} = 0$, $\text{RMS} = \sqrt{(9 + 16 + 16 + 9)/4} = \sqrt{12.5} = 3.54 \text{ m/s}^2$. Idle window: $\text{mean} = 0$, $\text{RMS} = 0.2 \text{ m/s}^2$. (b) RMS separates them (3.54 vs 0.2); the mean fails because symmetric oscillation averages to zero regardless of intensity. (c) Per axis: RMS; the 3 largest PSD peak amplitudes; the 3 corresponding peak frequencies; and 4 spectral power bins (0.1–0.5, 0.5–1, 1–2, 2–5 Hz) — 11 features $\times$ 3 axes = 33. The reduction addresses (1) computational complexity and (2) the large training-data requirement of high-dimensional inputs.

Solution 16. $TP = 38$, $FN = 17$, $FP = 3$, $TN = 42$; total = 100. $\text{ACC} = (38 + 42)/100 = 0.800$. $\text{TPR} = 38/(38 + 17) = 0.691$. $\text{TNR} = 42/(42 + 3) = 0.933$. $\text{PPV} = 38/(38 + 3) = 0.927$. $F_1 = 2(0.927)(0.691)/(0.927 + 0.691) = 0.792$ (equivalently $2 \cdot 38/(76 + 3 + 17) = 76/96 = 0.792$). Type II (false negatives, 17) dominates Type I (false positives, 3): the classifier misses positives far more often than it false-alarms.

Solution 17. (a) $(205 + 199 + 223 + 186)/1000 = 0.813$. (b) Up–Down positive: $TP = 186$; $FN = 21 + 8 + 3 = 32$ (Up–Down row); $FP = 46 + 32 + 34 = 112$ (Up–Down column); $TN = 1000 - 186 - 32 - 112 = 670$. $\text{ACC} = (186 + 670)/1000 = 0.856$; $\text{TPR} = 186/218 = 0.853$; $\text{PPV} = 186/298 = 0.624$; $F_1 = 2(0.624)(0.853)/(0.624 + 0.853) = 0.721$. (c) The Up–Down *column* collects large off-diagonal counts (46, 32, 34): the model over-predicts Up–Down, inflating FP and crushing precision (0.624) — so Up–Down has the lowest F_1 of the four classes even though its TPR is decent.

Solution 18. (a) MACs = $20 \times 33 + 10 \times 20 + 4 \times 10 = 660 + 200 + 40 = 900$. Parameters = $(660 + 20) + (200 + 10) + (40 + 4) = 934$. (b) `float32`: $934 \times 4 = 3,736 \text{ B} \approx 3.7 \text{ kB}$; `int8`: $934 \text{ B} \approx 0.9 \text{ kB}$. (c) $t \approx 900 \text{ cycles}/64 \text{ MHz} \approx 14 \mu\text{s}$ (perhaps $\sim 100 \mu\text{s}$ with activation and memory overhead) — versus a 2-s window: the network is $\sim 10^4 \times$ faster than real time. Feature extraction (the per-axis FFTs/PSDs) dominates the compute budget, and the sensing window dominates latency.

Solution 19.

```
import numpy as np, matplotlib.pyplot as plt
x = np.linspace(-6, 6, 400)
relu = lambda v: np.maximum(0, v)
sig = lambda v: 1/(1 + np.exp(-v))
for f, lab in [(relu, "ReLU"), (sig, "sigmoid"), (np.tanh, "tanh")]:
    plt.plot(x, f(x), label=lab)
plt.legend(); plt.xlabel("x"); plt.savefig("p19.pdf")

r = np.random.default_rng(0).uniform(-6, 6, 100)
assert np.allclose(np.tanh(r), 2*sig(2*r) - 1) # identity
eps = 1e-6; r = r[np.abs(r) > 1e-3] # avoid origin
num = (relu(r+eps) - relu(r-eps)) / (2*eps)
assert np.allclose(num, (r > 0).astype(float), atol=1e-6)
```

Both assertions pass: $\tanh(x) = 2\sigma(2x) - 1$ holds to machine precision, and the mask $(x > 0)$ equals the central-difference derivative of ReLU everywhere except at $x = 0$, where the derivative is undefined and assigned 0 by convention.

Solution 20. Using `train()` from the companion script over $\alpha \in \{0.001, 0.005, 0.01, 0.05, 0.5\}$ (representative run, seed 1): final costs $\approx 0.62, 0.090, 0.040, 0.0065$, and an erratic $O(10^{-1}\text{--}10^0)$ curve for $\alpha = 0.5$ that plateaus early and jitters. The stable regime is $\alpha \leq 0.05$: within it, larger

α converges monotonically faster (final cost spans ~ 2 orders of magnitude); $\alpha = 0.5$ overshoots — updates repeatedly jump across the cost valley — demonstrating the learning-rate trade-off.

```
for a in [0.001, 0.005, 0.01, 0.05, 0.5]:
    net, cost = train(X, Y, alpha=a)
    plt.loglog(np.arange(1, 1001), cost, label=f"a={a}")
    print(a, cost[-1])
plt.legend(); plt.xlabel("epoch"); plt.ylabel("C_total")
```

Solution 21. Sweeping $A \in \{5, 10, 20, 25\}$ at $\alpha = 0.01$: all four widths reach low cost on this 5-image problem; wider layers converge somewhat faster per epoch (more adjustable pathways), with diminishing returns from 20 to 25. Adding a second 20-node hidden layer does *not* materially improve the final cost and can converge slower early (more parameters to organize, deeper credit assignment). With only 5 training instances the single-layer model already fits perfectly, so extra capacity buys nothing and merely increases the parameter count — consistent with the guidance to use the smallest network that fits the problem, since surplus capacity invites overfitting on realistically noisy data (see Problem 24).

Solution 22.

```
import numpy as np, matplotlib.pyplot as plt
rng = np.random.default_rng(0); fs = 62.5
t = np.arange(0, 2, 1/fs)
def window(kind):
    n = 0.4*rng.standard_normal(t.size)
    return 6*np.sin(2*np.pi*1.0*t) + n if kind == "wave" else n
def feats(v):
    rms = np.sqrt(np.mean(v**2))
    V = np.abs(np.fft.rfft(v - v.mean()))**2
    f = np.fft.rfftfreq(v.size, 1/fs)
    return rms, f[np.argmax(V)]
for kind, c in [("wave", "tab:blue"), ("idle", "tab:orange")]:
    F = np.array([feats(window(kind)) for _ in range(50)])
    plt.scatter(F[:, 0], F[:, 1], c=c, label=kind, s=12)
plt.xlabel("RMS (m/s$^2$)"); plt.ylabel("peak freq (Hz)"); plt.legend()
```

The classes are cleanly linearly separable: “wave” clusters near (RMS ≈ 4.3 , peak ≈ 1 Hz) while “idle” sits near (RMS ≈ 0.4) with a random peak frequency; a vertical line at RMS ≈ 2 already separates them — even a single perceptron on these two features would classify perfectly, which is the point of feature extraction.

Solution 23.

```
import numpy as np
def confusion(actual, predicted, K):
    C = np.zeros((K, K), int)
    for a, p in zip(actual, predicted):
        C[a, p] += 1
    return C
def metrics(C, k): # class k as positive
    TP = C[k, k]
    FN = C[k, :].sum() - TP
    FP = C[:, k].sum() - TP
    TN = C.sum() - TP - FN - FP
    acc = (TP + TN) / C.sum()
    tpr = TP / (TP + FN); tnr = TN / (TN + FP)
    ppv = TP / (TP + FP)
```



```

    f1 = 2 * ppv * tpr / (ppv + tpr)
    return acc, tpr, tnr, ppv, f1
M = np.array([[205,10,1,46],[6,199,0,32],[9,17,223,34],[21,8,3,186]])
actual, predicted = [], []
for i in range(4):
    for j in range(4):
        actual += [i] * M[i, j]
        predicted += [j] * M[i, j]
C = confusion(np.array(actual), np.array(predicted), 4)
assert (C == M).all()
print([round(v, 3) for v in metrics(C, 1)])
# [0.927, 0.84, 0.954, 0.85, 0.845] -> matches the worked example

```

Solution 24. Sketch (representative behavior): with 200 noisy digits split 120/40/40, training accuracy climbs toward 100% while validation accuracy rises, plateaus (typically within the first few hundred epochs at $\alpha = 0.01$), and then drifts slightly down as the 625-parameter network begins memorizing pixel-flip noise in the 120 training images. Early stopping selects the epoch of peak validation accuracy (implement by saving the weight set whenever validation accuracy improves); evaluating that saved network once on the 40 test images gives the reported test accuracy, typically within a few percent of the validation peak. This implements the **early stopping** regularization countermeasure: halt training when performance on held-out validation data stops improving, and never touch the test set until the single final evaluation.

```

def accuracy(net, Xs, Ys):
    ok = sum(np.argmax(forward(net, x)) == np.argmax(y)
              for x, y in zip(Xs, Ys))
    return ok / len(Xs)
# inside the epoch loop:
# tr.append(accuracy(net, Xtr, Ytr))
# va.append(accuracy(net, Xva, Yva))
# if va[-1] > best: best, best_net = va[-1], copy.deepcopy(net)
# after training: report accuracy(best_net, Xte, Yte)

```